

Design and Evaluation of Area-Efficient Systolic Arrays for the Atalla Ax01 Accelerator

Nikhil Vaidyanath VIP 47921 nvaidyan@purdue.edu	Emmi Muninnimit VIP 47921 nmuninni@purdue.edu
Don Nguyen VIP 47921 nguye834@purdue.edu	Seongjoong Yim VIP 37900 yim13@purdue.edu
Myles Querimit VIP 27920 mquerimi@purdue.edu	Mixuan Pan VIP 37920 pan453@purdue.edu

Spring 2026

The feedback for the report is to be returned to:

Nikhil Vaidyanath
nvaidyan@purdue.edu

This project was mentored by the following Teaching Assistants:

Malcolm Lloyd Seib McClymont, mmcclym@purdue.edu
Sooraj Chetput Venkataraghavan, schetput@purdue.edu

Design and Evaluation of Area-Efficient Systolic Arrays for the Atalla Ax01 Accelerator

Nikhil Vaidyanath
VIP 47921
nvaidyan@purdue.edu
Seongjoong Yim
VIP 37900
yim13@purdue.edu

Emmi Muninnimit
VIP 47921
nmuninni@purdue.edu
Myles Querimit
VIP 27920
mquerimi@purdue.edu

Don Nguyen
VIP 47921
nguye834@purdue.edu
Mixuan Pan
VIP 37920
pan453@purdue.edu

Abstract

High-throughput and efficient GEMMs (General Matrix Multiplication) are crucial in AI accelerators, as the operation dominates machine learning workloads. Systolic arrays are widely used for this purpose due to their high parallelism and exploitation of data reuse. However, previous weight-stationary implementations in the Atalla accelerator incurred significant area overhead due to suboptimal buffering, inefficient processing elements, and global control wiring, increasing chip cost, and limiting scalability as array sizes grow. This work evaluates alternative systolic array architectures that target these sources of overhead. The first design, MEISSA (Multiplying Matrices Efficiently in a Scalable Systolic Architecture), eliminates input skewing and reduces buffering overhead. The second design, a Google TPU-inspired design, reduces accumulation cost by leveraging multi-input arithmetic units. Across synthesized designs on the FreePDK 45nm node, these approaches achieved a reduction in area relative to the baseline without sacrificing throughput. Potential drawbacks of these designs could include numerical error in the result due to differences in arithmetic units. To evaluate these trade-offs, we benchmarked the MEISSA, TPU-inspired, and standard designs across array sizes (4x4 to 32x32) for computation in BFloat16 (BF16) floating-point unit, using four-input and two-input adders where applicable. This comparison identifies that the two- and four-input (mixed) MEISSA variant provides the best area-performance tradeoff and informs the systolic array choice for the Atalla Ax01.

1 Introduction

1.1 Background

General matrix multiplication (GEMM) is a fundamental operation in many areas of engineering and science, namely graphics, signal processing, analytical solutions, and machine learning [1]. In machine learning applications, matrix multiplication is the predominant operation for modeling weighted connections between layers in neural networks [2]. ML models like Convolutional Neural Networks (CNNs) can represent their most compute-bound tasks, namely convolutions, as GEMMs. [3].

The large number of independent multiplications and accu-

mulations in GEMM makes it highly parallelizable. In this context, accumulation refers to the process of combining partial products into a final result, either through sequential additions or structured tree reduction across multiple operands. As such, dedicated hardware, such as systolic arrays, has been developed to exploit this property to accelerate computations. A systolic array is a network of processing elements (PEs) that rhythmically compute and pass data through the system [4]. These PEs are often implemented as Multiply-ACcumulate (MAC) units, and consist of a multiplication stage followed by the addition or accumulation of its partial products or sums. As the data moves through the PEs, a constant-time operation is performed: multiplication by weights followed by the accumulation of partial sums [4]. In practice, modern neural networks have many interconnections between layers, which are represented by large matrices, resulting in large GEMMs with dimensions on the order of thousands [2]. This means tiling – partitioning the computation into small chunks – is necessary to map the GEMM to the systolic array, which may be on the order of 4×4 to 128×128 . For a systolic array of dimension $N \times N$, an $M \times M$ GEMM where $M > N$ is partitioned into tiles, such that each weight tile is reused across $\frac{M}{N}$ input tiles, according to the principles of matrix multiplication. Weight and input-stationary dataflows exploit this reuse differently by prioritizing the reuse of weights or inputs, respectively. With high data reuse, the systolic array is less constrained by bandwidth, as only a single vector needs to be issued per instruction, that being weights or inputs. This reduction in bandwidth and nets saves power, area, and Vector Register File (VRF) bandwidth. In the case of constrained VRF capacity, Scratchpad or off-chip DRAM accesses may be required which can cost orders of magnitude more energy than the MAC operations they support [5].

1.2 The Atalla Baseline and Limitations

The Atalla Ax01 is a DSA (Domain Specific Accelerator) for machine learning workloads. Within the Vector Core subsystem, the systolic array serves as a dedicated functional unit for accelerating GEMMs. This approach allows the scheduler to issue GEMM instructions via the existing infrastructure, such as the VRF to store vector operands. Further integration details on the systolic array and Vector Core can be found in the Vector Core report. The existing baseline Atalla systolic array utilizes a weight-stationary data flow. In weight-stationary de-

signs, weights are loaded once into each processing element, remain fixed, and are reused across multiple input tiles, while activations and partial sums stream through the array. Thus, the reduced bandwidth comes at the cost of registers within every processing element. For the GEMM-heavy workloads that this report targets, weight reuse is a primary lever for reducing off-chip bandwidth at the dataflow level.

However, this approach comes with several inefficiencies. Input and output data typically must be temporally skewed as an inherent characteristic of the dataflow, requiring buffering. Furthermore, global stall signals are used to manage backpressure from the downstream GSAU module, which must be distributed to every PE in the array with high fanout. These mechanisms contribute significantly to chip area and routing congestion, which limits scalability to larger dimensions while maintaining performance.

1.3 Design Objectives

These limitations motivated the evaluation of alternative weight-stationary architectures.

Considering this, the criteria under which a new systolic array design was evaluated for the Atalla were specified as follows:

- Minimize array area while preserving high throughput, low computation latency, and meeting an 800 MHz timing target
- Support scalability across a wide range of array dimensions, from resource-constrained to data-center scale applications

1.4 Systolic Array Architectures

Modern systolic array designs differ primarily in the organization of the multiplication and accumulation stages. Two alternative weight-stationary designs in prior work are considered in this report alongside the baseline standard weight-stationary array: the MEISSA (Multiplying Matrices Efficiently in a Scalable Systolic Architecture) proposed by Georgia Tech researchers [1], and a Google TPU-patent-inspired design [6]. These architectures have distinct approaches to systolic array architecture, which address sources of overhead previously discussed, specifically the reduction of array area and maintaining latency and throughput. The weight-stationary dataflow was kept constant across architectures to be able to solely attribute any difference in PPA to architecture and design variation.

1.4.1 MEISSA

The primary novelty of MEISSA lies in decoupling the multiplication and accumulation stages that are traditionally combined within a multiply-accumulate (MAC) unit [1]. Instead of a grid of MAC processing elements, a MEISSA systolic

array is organized as a grid of multipliers feeding into dedicated column-wise adder trees [1]. As a result, partial products are reduced through the tree structure rather than sequentially across processing elements. According to the authors of MEISSA, this transformation reduces accumulation latency from linear to logarithmic depth and processing latency to scale sublinearly with array dimensions [1]. Additionally, input activations no longer need to be skewed (i.e., injected as parallelograms) to align with processing elements [1]. This eliminates the need for input buffers, which are significant contributors to area overhead in conventional weight-stationary designs.

1.4.2 TPU-Style

In contrast, TPU-style (Tensor Processing Unit) systolic arrays retain the conventional grid structure of MAC processing elements, thus requiring skew and de-skew of input and output data. However, the design uses multi-input arithmetic units to process multiple operands in a single unit. As such, area and latency improvements can be found in this implementation. Some TPU designs, such as the Google-patented design, also incorporate a fast weight-loading mechanism [6], which is not explored in this work.

2 Design Choices

2.1 Adder Arity in Accumulation: Two vs. Four

When implemented using two-input arithmetic units, reduction requires multiple sequential addition stages, and thereby processing units, to combine several operands, either temporally within MAC units or spatially in a reduction (adder) tree. As the array size increases, the number of operands grows, leading to greater area cost and accumulation stages.

An alternative is to use multi-input arithmetic units, such as four-input adders, to combine more operands within a single unit. According to the adder team’s results, a single four-input adder is $2.423\times$ more area-efficient than an equivalent sequence of two-input additions. Additionally, it decreases the number of accumulation stages required and overall array latency to produce a result.

2.2 Credit System for Handling Stalls

The previous design used a global stall signal to halt the systolic array pipeline in response to the backpressure of the GSAU. Although it is necessary, it comes with the cost of increased routing congestion and area, as they have high fanout, and must be distributed across all processing elements in the array. Furthermore, as the array size increases, the cost of distributing the global signal grows with the number of PEs. To address this issue, a credit system was developed in the new design to replace the global stall signal. The credit system tracks available output buffer capacity, allowing vector issuing and computation to continue as long as there is

sufficient space. This eliminates the high signal fanout and allows the pipeline to remain active under backpressure, improving throughput and scalability. The main costs of this approach are the additional control logic and registers for the credit counter. No additional alternatives were considered as this solution solved the issue with relatively small costs in the area.

2.3 Memory Structures

2.3.1 SRAM Usage

The previous design used flip-flops in buffers, which stagger inputs and de-stagger outputs from the systolic array. However, as the array size increases, the area cost of flip-flops quickly becomes significant compared to that of other technologies such as SRAM or DRAM. To address this issue, SRAM was used as on-chip memory for all three designs. A typical single SRAM cell uses only 6 transistors compared to the 18 or more in a D flip-flop [7], effectively increasing the storage cell density per unit area. The tradeoff of SRAMs is comparatively higher latency, specifically for reads in our designs. Nevertheless, because the systolic array designs are already optimized for throughput, the tradeoff of reduced area at the cost of modest additional latency is favorable. Other memory architectures, such as DRAM, were rejected as they can throttle throughput, latency, or have impractical fabricability.

2.3.2 Buffer Sizing

For the input buffer in TPU and standard systolic arrays, the input buffer’s size must be larger than $N \times N$ to skew the input. For the output buffer, all architectures need a minimum buffer depth of $N + P$ where P is the pipeline depth. This is necessary for back-to-back GEMMs, where the N slots may be used for output de-skewing. The nature of the ready/valid protocol allows the consumer to stall at any point, meaning the output buffer spacing must account for the possibility of stalls. With $N + P$ spacing, the array would still stall on every cycle that the consumer module is not ready to accept the output. For these reasons, it’s likely that the output buffer will be sized to greater than the minimum requirements. To get a better area estimate, all buffers depths rounded their depths from the minimum required to the next power of two, which fulfills an OpenRAM requirement.

2.4 FP32 Accumulation

All three architectures consume BF16 inputs and produce BF16 outputs, but perform accumulation in FP32. This follows the same mixed-precision structure adopted by commercial tensor core implementations such as NVIDIA’s Ampere architecture [8]. The previous design exclusively supported FP16, with the accumulator matching the operand format end-to-end. However, the narrow mantissa bits introduced rounding errors across the chain of MAC operations. Since each MAC addition operation contributes to approximately one ULP (Unit in Last Place) of error, the dominant

source of error was due to rounding rather than the width of the operand mantissa. This error obscures the accuracy differences between the 3 architectures when comparing the different addition/reduction orders (e.g., reduction tree vs group addition). Widening only the accumulator to FP32’s 23-bit mantissa can address this issue without altering the I/O format. Using BF16 instead of FP16 for inputs gives us architectural benefits in adders. BF16 shares the same 8-bit exponent of FP32, operand conversion reduces to only zero-padding. This property also allows the Wallace-Tree multiplier inside each multiplier to forward its full 16-bit mantissa product directly into the adder, whereas same-format accumulation would discard roughly half of the output. Using FP32 input was rejected due to the increased bandwidth. FP32 operands would double the memory bandwidth and buffer footprint, with prior empirical work demonstrating that full FP32 accumulation would lead to no commensurate accuracy gain compared to BF16 operands with FP32 accumulation in many ML training workloads [9]. The current cost of the chosen approach is a 32-bit partial-sum register in each PE, widening the vertical partial-sum path, and an additional FP32 to BF16 conversion stage at the end of each column before writing to the systolic array’s output buffer.

3 Micro-architecture

3.1 Common Features

This subsection describes the shared micro-architectural features across all evaluated systolic array architectures. In addition to the modules described in the following subsections, all architectures implement a common module interface to maintain consistency between designs to support integration with the Vector Core. This included ready/valid handshaking, which was implemented to abstract away the latencies of the systolic array, allowing each array architecture to be a black box for drop-in replacement and faithfully perform *ceteris paribus* PPA comparisons. As these are weight-stationary architectures, weights are first streamed into the array and stored locally in registers at each multiplier during computation. Each design also supports parameterizable array dimensions and arithmetic unit latencies. This functionality was necessary for porting the designs to different environments, such as a 2-cycle adder on MITLL 90nm to a single-cycle adder on FreePDK45.

3.1.1 Processing Elements

Across the three architectures, the arithmetic primitives inside each processing element, the BF16 multiplier and the FP32 adder, are identical; what differs between them is how those primitives are arranged, namely that of a grid PE formation (Standard), a multiplier grid with associated adder trees (MEISSA), or grouped into a PE grid with many-input additions (TPU). Constant arithmetic primitives further contribute towards design uniformity, ensuring faithful *ceteris paribus* comparison between architectures.

Both primitives implement flush-to-zero (FTZ) on outputs and denormal-as-zero (DAZ) on inputs, so any subnormal operand is treated as signed zero before arithmetic operations, and any subnormal result is replaced with signed zero before being passed downstream. The motivation is to improve area efficiency, as it reduces the amount of logic needed to handle subnormal values. Full IEEE 754 subnormal support requires a leading-zero detector, a variable-width left shifter to normalize the significand, and additional exponent logic in every adder and multiplier [10]. When this is replicated across all N^2 PEs of the array, this overhead is substantial, and initial attempts to include subnormal handling yielded inaccurate outputs that were difficult to justify for a regime with significant area cost.

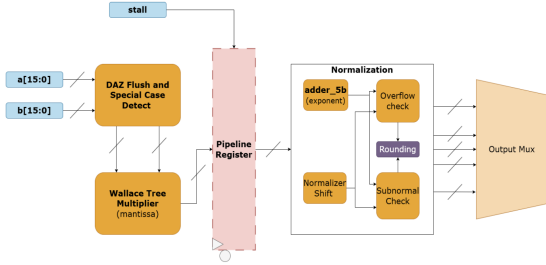


Figure 1: Multiplication Unit Design.

3.1.2 Credit System

The credit system is common to all three architectures. A counter is used to track the remaining output buffer capacity on a vector. The counter decrements when a vector is issued to the array, and increments when a vector is read from the output buffer. When the counter reaches zero, new vector issues are prevented until a credit is returned. The credit system does not stop or freeze the dataflow allowing already issued vectors to complete. This allows the systolic array to continue processing inputs until all credits are consumed, even if the GSAU propagates a stall from the output buffer.

3.1.3 Input/Output Skew Buffers

The Standard and TPU architectures require staggered inputs, and all three architectures produce staggered outputs. SRAM-based input and output buffers handle this staggering and de-staggering. In the input buffer, all lane write pointers increment together, storing each incoming vector at the same address in each lane. Read pointers advance independently in each lane, controlled by enable signals. The output buffer inverts the pointer movement, where write pointers advance independently per lane to absorb staggered outputs, while the read pointers increment together to read out complete vectors. Both pointers wrap around when reaching the end of the buffer. The buffers have a minimum size of $N + D$ where N is the array dimension, and D is the pipeline depth, defined as the number of cycles from input to output availability.

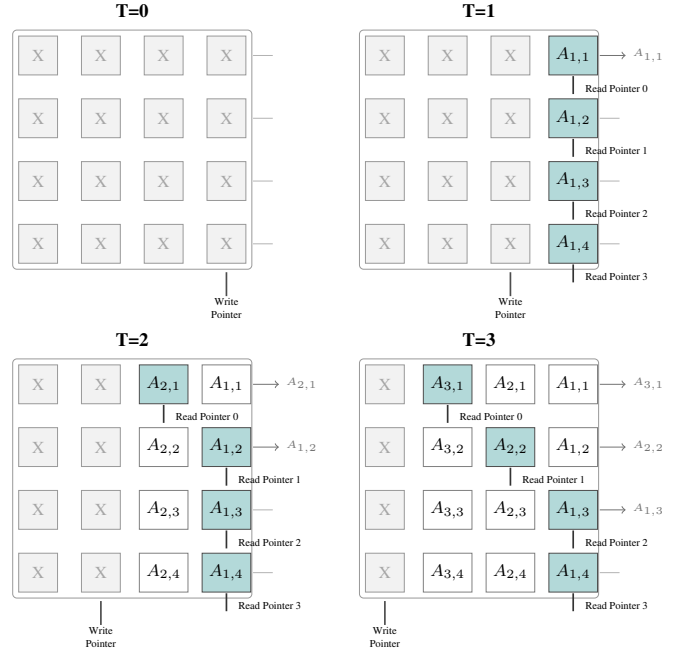


Figure 2: SRAM input buffer state progression for a 4×4 buffer. Each column stores one input vector; the write pointer advances left with each write. Read pointers advance independently per row, producing the staggered reads required by the systolic array. Teal cells indicate the current read address for each row.

3.2 Standard Weight-Stationary Architecture

3.2.1 Overview

The Standard architecture is a classical Kung-style weight-stationary systolic array [4] and serves as our baseline against which the MEISSA and TPU-style implementations are measured. Unlike MEISSA, which separates multiplication and accumulation into a dedicated multiplier grid followed by adder trees, and unlike the TPU-style design with grouped additions, the Standard design performs one multiply and one accumulate per cycle at each PE (processing element) in a uniform grid. The standard array is a parameterizable $N \times N$ grid of identical MAC processing elements, with both MAC latency and numeric mode (BF16/BF16 or BF16/FP32) exposed as compile-time parameters.

3.2.2 Computation Flow

Partial sums enter the top of the array as zeros, and any pre-existing cross-tile contributions are folded in externally by the Atalla vector core, keeping the array stateless across tiles. Skew buffers are also utilized in combination with the control logic to ensure proper dataflow pre- and post-compute.

3.2.3 Processing Element

In regards to BF16/FP32 accumulation, within each PE, there is a BF16 multiplier, which utilizes a Wallace-Tree for the mantissa portion, FP32 accumulator, and registers for weights, partial-sums, and activation pass through. The PE is pipelined

in 2 stages, with the first latching in the multiplier output as FP32, and the second performing the FP32 accumulation. With the weight-stationary design choice in mind, the weights reach the array through the same horizontal path that later carries in the activations. Each weight vector is shifted in during weight loading before computation begins.

3.2.4 Standard Systolic Array Control Logic

Control is distributed at the array boundary, with the controller issuing a single per-row enable pulse and a global weight-load pulse, which propagate left-to-right across the rows of PEs, through the use of shift registers.

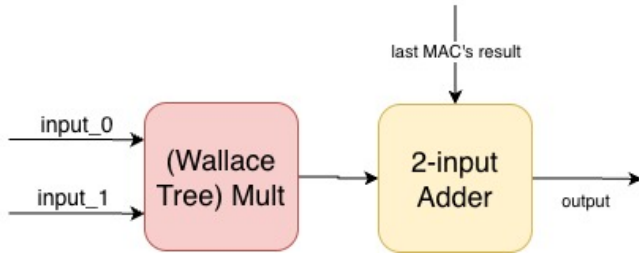


Figure 3: Standard MAC Unit.

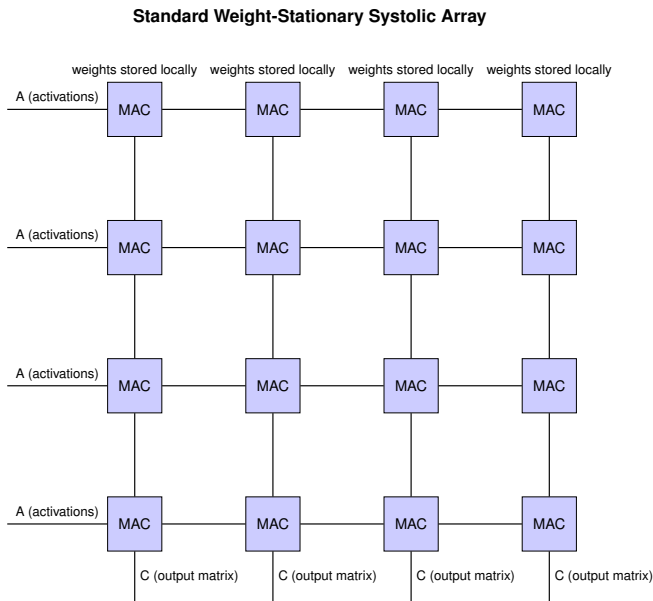


Figure 4: Standard Weight-Stationary Systolic Array Microarchitecture and PE Pipeline.

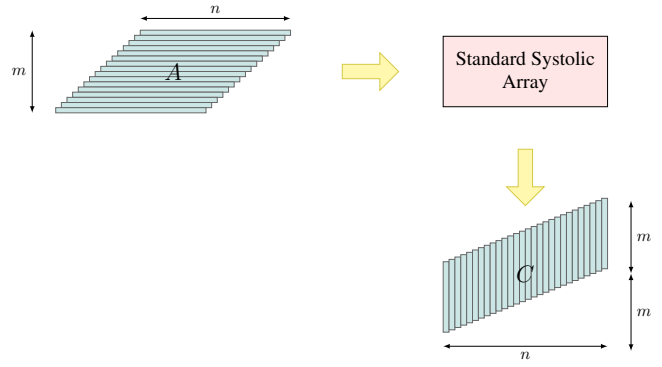


Figure 5: Standard Weight-Stationary Systolic Array Input and Output Staggering.

3.3 MEISSA Architecture

3.3.1 Overview

Traditionally, systolic arrays intertwine multiplication and addition into an integrated MAC. MEISSA, on the other hand, separates this paradigm into a distinct grid of multipliers and adder trees associated with each column.

3.3.2 Computation Flow

At each clock cycle, an input vector is injected into the multiplier grid from column 0 and propagates across the array column by column. Similar to the standard architecture, each multiplier computes the product between the incoming input value and its locally stored weight, producing a set of partial products every cycle that inputs are available. These partial products are stored in pipeline registers and forwarded to column-wise pipelined adder trees, which reduce them to a single output per column. Since the generation and reduction of partial products are pipelined in the multiplication grid and adder trees, the need for temporal input skewing is eliminated. Partial products and sums are maintained in FP32 precision during multiplication and reduction, with conversion to BF16 occurring through a reducer after the reduction stage. It is essential to reduce before storing such that the output buffer needs to only store BF16 values rather than FP32 values, which are double the size. After the systolic array's pipeline is filled, the array produces one output per cycle.

3.3.3 Adder/Reduction Tree

The adder tree in the array has two configurations: a two-input adder tree and a mixed four-input/two-input adder tree. The mixed configuration was investigated to reduce the adder tree area by using four-input adders where possible. In our implementation, four-input adders are 2.423 $\times$ more area-efficient than three cascaded two-input adders, according to metrics provided by the Adder Team. The structure (i.e., the number of layers and adders per reduction layer) of the adder tree depends on the array dimensions. For example, a 16 $\times$ 16 array can be reduced using only four-input adders, as the array dimension is a power of four. In contrast, a 32 $\times$ 32 array cannot

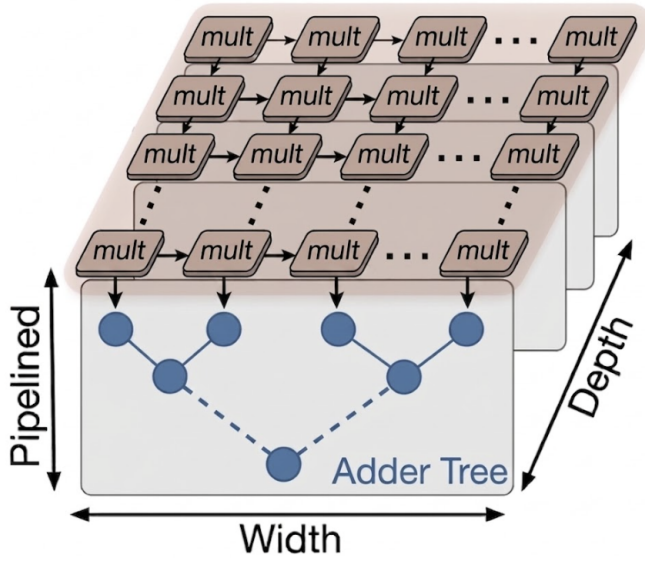


Figure 6: MEISSA multiplier layers and adder-tree depth/width visualization derived from [1].

be fully reduced using only four-input adders and therefore requires a final two-input reduction stage, as 32 is not a power of four. As a result, there is a function that determines whether a pure four-input reduction tree is possible or if a mixed structure is needed that runs at compile time. The number of reduction layers also differs between configurations, as a two-input reduction tree has depth $\log_2(n)$, while a four-input tree reduces this to $\log_4(n)$, with N being the array dimension. Accordingly, the adder tree structure and pipeline depth based on array size and adder type were parameterized.

3.3.4 MEISSA Control Logic

Outputs from the adder tree are fed into the output buffer, which de-skews outputs before sending them to the GSAU and holds outputs in the case of a stall. To indicate when the outputs are valid, a shift register propagates valid signals alongside the data. Input valid signals are shifted through the pipeline stages and used to generate output valid signals once the corresponding data has been fully processed.

3.4 TPU-Style Architecture

3.4.1 Overview

The TPU-style Systolic array extends the standard weight-stationary architecture by performing four multiplications and two accumulations in each PE per cycle once the pipeline is full. [6], rather than a single multiplication and accumulation. The architecture supports $N \times N$ matrices where N is a multiple of four, using MACs in an $\frac{N}{4} \times N$ grid. MAC latency and numeric modes (FP16/FP16 or BF16/FP32) are compile-time parameters. Input and output buffers, combined with control logic, handle skewing of inputs and de-skewing of outputs.

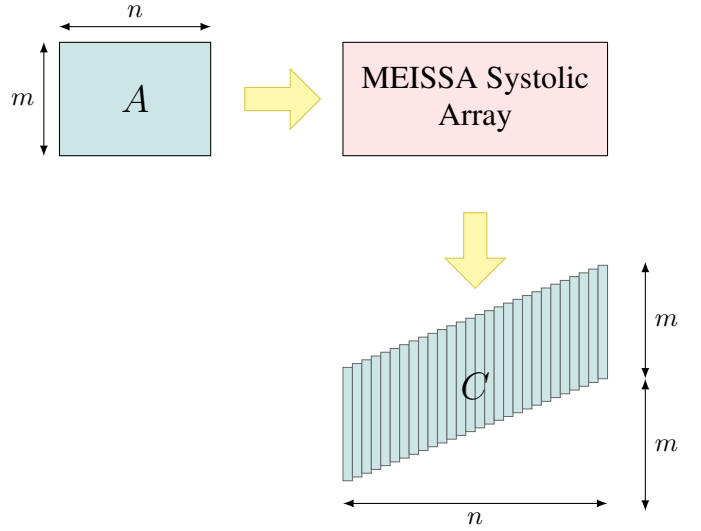


Figure 7: MEISSA Systolic Array Output Staggering (Vertical Column Emission).

3.4.2 Computation Flow

In operation, weights are first shifted into the systolic array column-wise, beginning with the rightmost column of the weight matrix. The input vectors are then received from the GSAU and stored within the input buffer. On a vector issue, the first four elements of the input vector will be sent to the systolic array, followed by the subsequent four elements X cycles later, where X is the two-input adder latency. Each input will propagate across the columns of the systolic array. When entering a MAC, each multiplier computes the product of the input value and the weight stored in each multiplier's register, and the resulting partial sum is accumulated and output to the row below. The TPU's dataflow is the same as the standard systolic array design, where input flows from left to right and partial sums flow from top to bottom. Each column of the systolic array generates a single element of the output vector. Shifting between PEs takes a clock cycle, making the resulting output vector skewed since the leftmost columns receive inputs earlier than the rightmost columns.

3.4.3 TPU MAC

The MACs in the TPU design consist of four multipliers in which the inputs and weights are BF16 formats. The FP32 results from the multipliers are summed by a four-input adder and accumulated with the incoming partial sum using a two-input adder, resulting in the FP32 partial sum output.

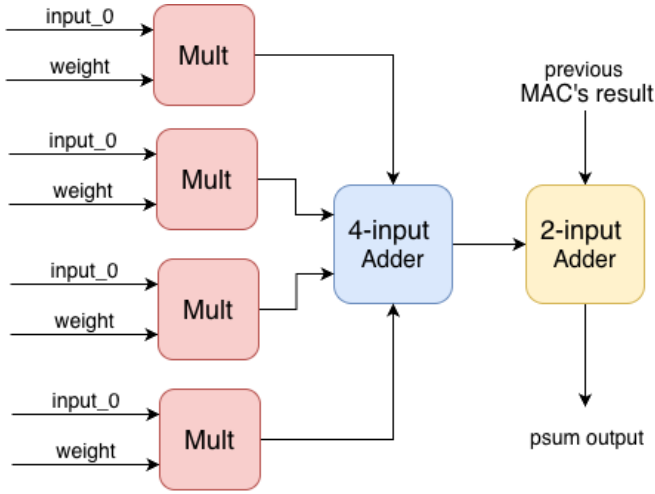


Figure 8: TPU MAC Unit.

3.4.4 Control Logic

To stagger inputs to the MAC grid and de-stagger outputs, a shift register is utilized as an issue timer. The shift register is divided into sections for the input buffer read signals and output buffer write signals. When a vector is issued to the array, a 1 is shifted in at one end and propagates through the register each cycle. The register is sectioned into three sections, the first being the input stagger window, where each bit position corresponds to a line's read enable for the input buffer, causing lanes to be read cycles apart. The second section spans the MAC pipeline latency, accounting for the cycles data spends in the compute stage. The third section spans the pipeline depth, where the same bit's arrival at each output position triggers the corresponding output buffer write enable, absorbing the skewed results as they are produced from the array. This allows both input staggering and output de-staggering to be driven from a single shift register. The credit system is utilized to communicate with the GSAU and control writing vectors to the input buffer and reading vectors from the output buffer. It also controls the issuing of new vectors from the input buffer to the MAC grid.

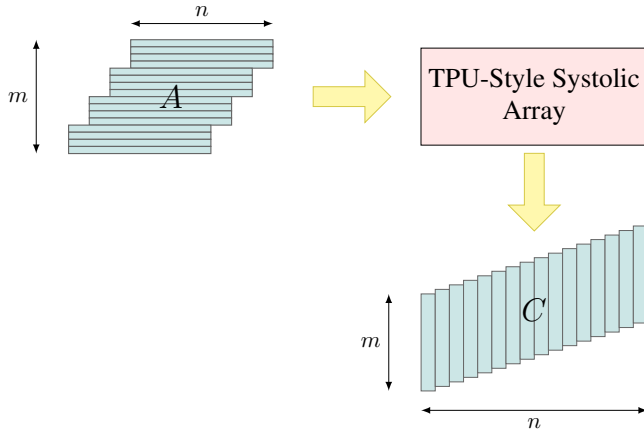


Figure 9: TPU Systolic Array Input and Output Staggering.

4 Methodology

4.1 Processing Elements

4.1.1 Arithmetic Primitive Verification

The correctness of the arithmetic primitives was validated extensively prior to integration. Both the BF16 and FP16 multiplier and adder were verified against over 1 million randomized test vectors, in addition to directed edge-case tests that explicitly targeted boundary conditions such as overflow, underflow, zero propagation, sign handling, and rounding corner cases. Reference outputs were generated using TestFloat and Berkeley SoftFloat, ensuring compliance with IEEE-754 semantics under the adopted FTZ and DAZ policies. Across all test cases, the implementations achieved 100% functional correctness.

4.1.2 Arithmetic Primitive Synthesis

The designs were synthesized using standard-cell libraries at multiple technology nodes to evaluate area, power, timing, and scalability. The arithmetic units were optimized for high-frequency operation, achieving timing closure at 600MHz in 90nm and 800MHz in 45nm. Area efficiency was a primary design objective, and the resulting implementations demonstrate a 34.4% area reduction compared to baseline designs, largely due to the elimination of subnormal handling logic through FTZ and DAZ. Synthesis results were gathered using Cadence Genus.

To characterize timing robustness and guide architectural design decisions, a shmoo analysis was performed across a range of clock frequencies. As shown in fig. 10, the synthesized area of both FP16 and BF16 adders exhibits non-monotonic behavior as frequency increases, reflecting synthesis tool optimizations such as gate sizing and restructuring under varying timing constraints. Across the entire frequency range, the BF16 adder consistently maintains a lower area footprint than the FP16 adder, with an average reduction of approximately 25–30%. This improvement is primarily driven by the reduced mantissa width and simplified datapath logic.

These observations were used to select realistic operating points for the arithmetic units and to inform system-level frequency targets for the systolic array and Atalla accelerator, where replication across N^2 processing elements amplifies both performance and hardware cost. The same arithmetic primitives are also instantiated in the vector core, though the systolic array dominates the logic-area portion of the accelerator's PPA (excluding scratchpad SRAM). By grounding these decisions in empirical synthesis data, the reported performance and area metrics more accurately reflect achievable hardware characteristics.

4.2 Systolic Array

4.2.1 Verification

Functional correctness of each architecture was verified against 500,000 random test cases. Random input and weight

Adder Area Shmoo Result

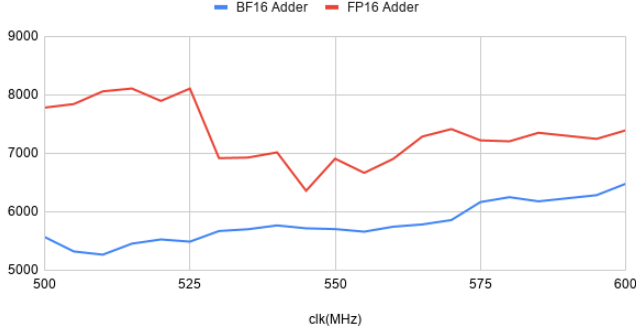


Figure 10: Adder area versus clock frequency at 90 nm for FP16 and BF16 implementations.

matrices were generated with a maximum exponent capped at 19. This number was chosen since it would be enough of a stress test, as GPT 2 weights have a maximum exponent of 4. Simulations for each architecture were developed using Berkeley SoftFloat [11] and a custom four-input adder model to emulate exact hardware behavior. All 3 implementations achieved 100% functional correctness.

To verify the accuracy of each architecture, we compared the result with the golden model using an accumulator that expands FP32 into a much larger integer format. An integer accumulator was used due to the non-associative nature of FP32 addition. In the process of many additions, which is prevalent in systolic arrays, substantial exponent differences between operands can cause catastrophic cancellation, losing significant numbers in the result [12]. The integer format used in the reference model mitigates this concern through its capacity to utilize all precision bits in the FP32 range, ensuring an accurate result to compare hardware results with. To measure the error, metrics for Unit in Last Place (ULP) error were gathered. ULP error is a measure of the distance between the reference number and the obtained number [13]. Average, maximum, and percentage of cases with any error were gathered over 100,000 random test cases to evaluate correctness.

4.2.2 SRAM/Buffers

OpenRAM Elmore Modeling was used to estimate the area of each bank, which was totaled to gather the total area of the buffer. Due to the nature of the software, the depth of the SRAM was locked to powers of two, and as such, all depths were rounded to the next greatest power of two.

5 Results and Discussion

Table 1: ULP analysis of 32×32 designs over 100,000 random test cases.

Architecture	Average ULP Error	Maximum ULP error	Percentage of ULP errors (%)
Standard	0.04608	115	0.004
MEISSA (mixed)	0.04039	115	0.003
TPU	0.06023	113	0.005

As shown in Table 1, all three designs showed low average and percentage of ULP errors and similar maximum ULP error. MEISSA achieved the lowest average and percentage ULP error, giving more accurate results than the standard design. The TPU-inspired design has the lowest maximum ULP error, but has worse average results due to a higher proportion of results that have errors. Given that MEISSA’s maximum error is only slightly higher than TPU, we have ascertained that MEISSA is the most accurate systolic array design in our testing.

Table 2: PPA metrics for 16×16 designs synthesized at 800 MHz.

Metric	MEISSA (two-input)	MEISSA (mixed)	TPU	Previous
Total Area (mm ²)	1.703	0.8159	1.030	2.140
Total Area w/ buffers (mm ²)	1.823	0.9354	1.269	2.379
Total Power w/o buffers (W)	2.059	0.9948	0.7255	2.377
Instr Latency (cycles)	23	25	29	37

It can be seen from Table 2 that mixed MEISSA achieved the smallest total area with and without buffers, while the baseline design had the largest. However, the TPU has the lowest power consumption.

Table 3: Area scaling for sizes from 4×4 to 32×32 designs synthesized at 800 MHz.

Metric	MEISSA (two-input)	MEISSA (mixed)	TPU	Standard
4×4 to 16×16 ($\sqrt{\text{times}}$)	4.195	4.197	4.625	4.253
16×16 to 32×32 (times)	4.186	4.251	4.0864	4.078
Average (times)	4.192	4.215	4.438	4.194

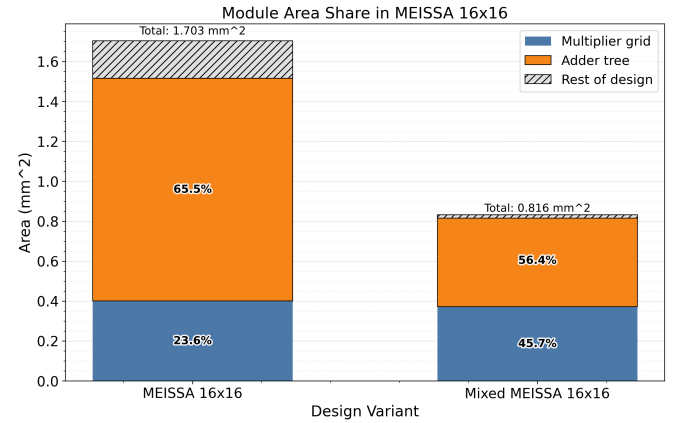


Figure 11: Area of adder tree for MEISSA and mixed MEISSA at 16×16 array size.

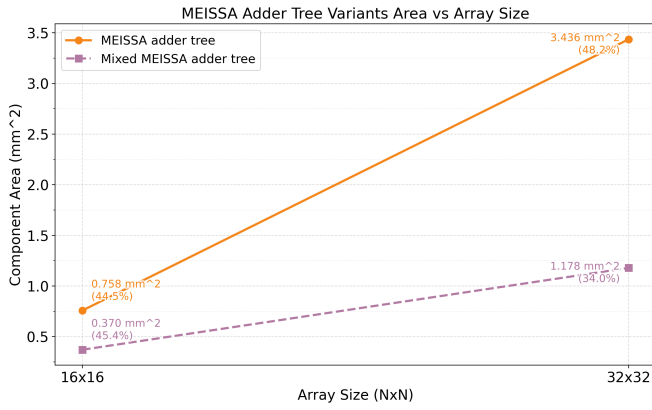


Figure 12: Area of adder tree for MEISSA and mixed MEISSA at 16x16 and 32x32 array size.

The graph above, fig. 12, illustrates the impact of increasing array size on the adder tree of the MEISSA variants. The mixed MEISSA design exhibits more favorable adder tree scaling (i.e., less steep growth).

The difference in adder tree area shown in fig. 11 accounts for the total area difference of mixed and non-mixed MEISSA observed in table 2. This is expected, as the adder tree unit is the only affected unit by the four-input adder substitution, while the multiplier grid remains unchanged between the two designs. Consequently, this reduction in array area is due to the adder tree and the four-input adder implementation, which is 2.423 $\times$ more area-efficient than an equivalent two-input adder structure.

These results indicate that the reduction structure was the main contributor to the area in MEISSA in the non-mixed implementation. As such, the relative contribution of the multiplier grid becomes more prominent, and further area improvements may require optimizations to the multiplier array.

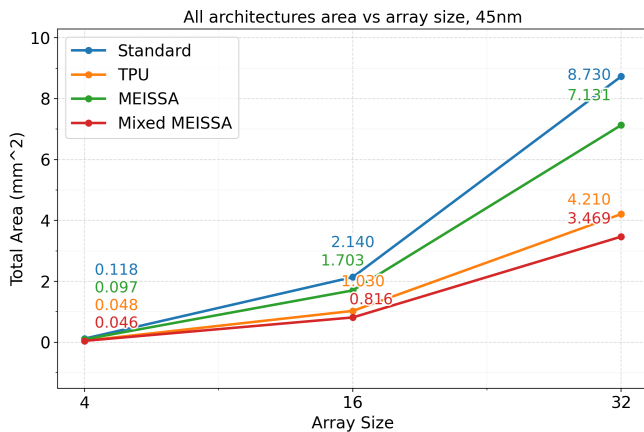


Figure 13: Systolic array area trends for array sizes 4x4, 16x16, and 32x32 of evaluated architectures.

The graph above, fig. 12, illustrates the impact of increasing array size on the adder tree of the MEISSA variants. For all array sizes, mixed MEISSA remains the most area-efficient design.

As seen in table 3, all four designs have an area scaling factor around 4 $\times$, which is what is expected when scaling the array dimension by 2 $\times$. Of the four architectures, the two-input MEISSA design has the lowest average area scaling at 4.192 $\times$, and the TPU design has the lowest average power scaling at 3.890 $\times$, but the largest area scaling at 4.438 $\times$. This suggests that the two-input MEISSA exhibits the most favorable area scalability, since its total area scaling factors better adhere to the ideal quadratic scaling than other architectures. For designs with larger fan-in, like those that use the four-input adder, this could be a reason for worse average scaling than those that use the two-input adder. Because systolic arrays are generalized to be highly parameterizable and regular, the scaling for each implementation shows the architecture's impact on those tenets.

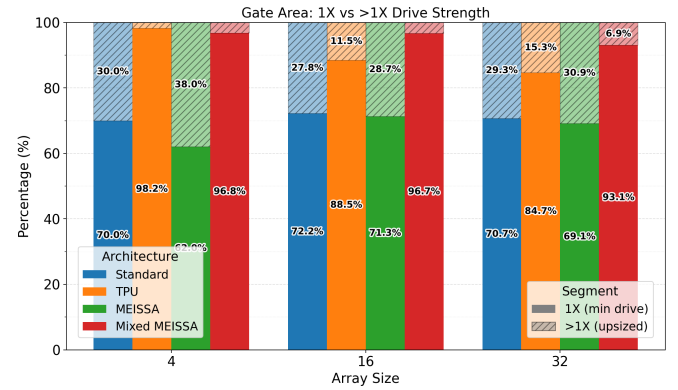


Figure 14: Systolic array gate drive strength breakdown for all architectures (45nm) at 4, 16, and 32 array dimensions.

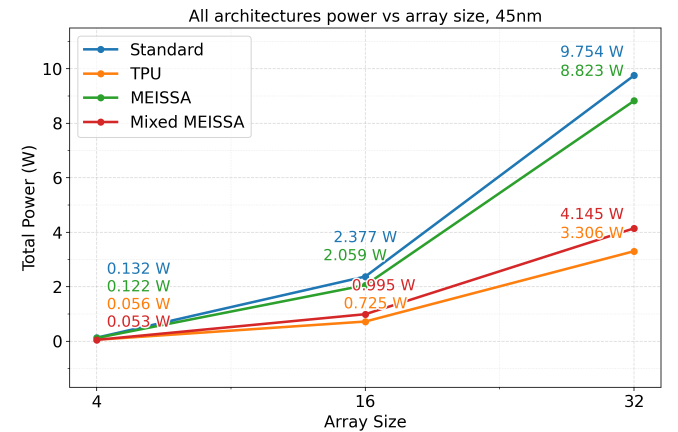


Figure 15: Systolic array power trends for array sizes 4x4, 16x16, and 32x32 of evaluated architectures.

From table 2, fig. 13 and fig. 15, it is evident that the TPU and mixed MEISSA designs are far better than the Standard and two-input MEISSA designs in PPA in terms of raw numbers. This improvement can be attributed to the area and power efficiency of the four-input adder compared to the two-input adders in the other designs. However, at this timing constraint of 800 MHz, some designs at certain array sizes did not meet

timing closure. This is further exemplified by the much larger proportion of gate count and area taken by the designs that predominantly use the two-input adder seen in fig. 14. The conclusion we can draw from this is that the two-input adder is struggling to synthesize at this frequency. The two-input adder is a single-cycle design, while the four-input adder takes three cycles; assuming the four-input adder was perfectly pipelined, this means there is significantly less logic in the four-input adder compared to the equivalent three two-input adders.

6 Limitations

The biggest limitation of this work is the testing methodology. The systolic array designs emphasize the use of short wires and parameterizable computing units, suggesting that they should be very scalable. However, we were unable to gather enough data at sizes larger than a matrix tile size of 32×32 elements to conclusively confirm this trend due to limited computational resources. Additionally, we observed inconsistencies in how arithmetic units were synthesized across different configurations, leading to non-uniformity in timing closure behavior that cannot be explained by the architectural design alone.

As mentioned, the previous designs were implemented using global wires. This motivated the use of a buffer space tracking system to reduce wire congestion. However, to accurately evaluate the impact of this improvement, a full place-and-route flow would be necessary, such that RC effects are fully accounted for, and congestion is realized in the layout PPA figures. Because the collected metrics are derived solely from synthesis, the true effect of the buffer space tracking system on congestion and post-layout PPA remains uncertain.

Regarding power estimates, it's important to note that SRAM can use a significant portion of the total power budget of a chip [7]. As a result, the power numbers which are primarily focused on logic, likely miss a significant portion of the power allocated to SRAM. The use of Elmore modeling in SRAM generation is itself an approximation of a more accurate SPICE-based model. Future work should focus on using SPICE-based models of multi-bank SRAM configurations, integrating them into the synthesis tool flow to improve the accuracy of PPA numbers.

FreePDK45, a CMOS 45nm predictive PDK, was used in the synthesis of the designs. This PDK includes the Nangate cell library, which contains 38 functions and over 100 cell variants [14]. This is likely far less than the number of functions and cells present in an industry-standard fabricable cell library. As a result, the synthesis tool may have fewer options to optimize the design to meet timing at difficult constraints.

Another important limitation lies in the handling of subnormal values within the arithmetic units. To improve area efficiency, both flush-to-zero (FTZ) and denormal-as-zero (DAZ) policies were adopted. While this significantly reduces the complexity of the datapath and avoids the overhead associated with normalization hardware, in the context of systolic array, where accumulation occurs across many cycles, this behavior can introduce loss of precision.

Regarding the design itself, the Google TPU patent-based design was not implemented with the accelerated weight loading scheme. Additionally the design did not include the transpose buffers, both of which were specified in the patent design diagrams [6]. The result of this is that there may be reduced performance compared to the Google TPU design in some workloads, and the trade-off is that the implemented design saves the area and power of the extra required hardware.

Future projects to further explore these architectures include PPA from place-and-route, implementing multi-format designs (i.e., supporting both BF16 and FP8), and using VCD/SAIF dumps from full system simulations to generate accurate power and performance numbers that model the actual observed activity under the target workload.

7 Conclusion and Future Work

This work evaluated whether two alternative weight-stationary architectures, MEISSA and a TPU-style design, could reduce the area cost without sacrificing throughput or numerical accuracy. MEISSA uses a decoupled multiplier grid and pipelined adder trees to remove the overhead of an input buffer. The TPU design exercises the use of four-input accumulation in place of two-input accumulation. Mixed MEISSA combines the architectural benefits of MEISSA and the predominant use of the four-input adder like TPU.

At 16×16 on FreePDK 45nm synthesized to 800 MHz, mixed MEISSA reduced total array area from 2.140 mm^2 to 0.816 mm^2 while also cutting power from 2.377 W to 0.995 W. The TPU design achieved the lowest power at larger array sizes (3.306 W at 32×32 versus 4.145 W for mixed MEISSA), but at the cost of higher area. On numerical accuracy, mixed MEISSA achieved the lowest average ULP error at 0.04039 across 100,000 random 32×32 test cases. All four designs exhibited approximately $4 \times$ area scaling when doubling array dimension.

The primary cause of mixed MEISSA's area advantage is the elimination of the SRAM input buffer, combined with the reduced adder tree footprint from four-input reduction. The TPU's power advantage at larger sizes reflects the same four-input adder efficiency, though it retains the input buffer overhead that MEISSA avoids.

These results support mixed MEISSA as the preferred architecture for integration into the Atalla Ax01. Confirmation of these conclusions at array dimensions beyond 32×32 remain as future work.

References

- [1] B. Asgari, R. Hadidi, and H. Kim, "Meissa: Multiplying matrices efficiently in a scalable systolic architecture," in *MEISSA: Multiplying Matrices Efficiently in a Scalable Systolic Architecture*, 10 2020, pp. 130–137.
- [2] M. Sinclair, "Gemms: (one of) the building blocks of fast machine learning." [Online]. Avail-

able: <https://pages.cs.wisc.edu/~sinclair/presentations/msinclair-lsu-gemms.pdf>

- [3] S. Hadjis, F. Abuzaid, C. Zhang, and C. Ré, “Caffe con troll: Shallow ideas to speed up deep learning,” in *Proceedings of the Fourth Workshop on Data Analytics in the Cloud*, ser. DanaC’15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: <https://doi.org/10.1145/2799562.2799641>
- [4] H. T. Kung and C. E. Leiserson, “Systolic arrays for (vlsi),” Carnegie-Mellon University, Department of Computer Science, Pittsburgh, PA, Tech. Rep. CMU-CS-79-103, 1978.
- [5] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [6] A. E. Phelps and N. P. Jouppi, “Low latency matrix multiply unit,” U.S. Patent US 2020/0 327 186 A1, Oct. 15, 2020, patent Application Publication.
- [7] D. M. Harris and N. H. E. Weste, *CMOS VLSI Design: A circuits and systems perspective*, 4th ed. Pearson/Addison Wesley, 2004.
- [8] NVIDIA Corporation, “NVIDIA A100 Tensor Core GPU Architecture: Unprecedented Acceleration at Every Scale,” 2020, white paper, <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>.
- [9] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, J. Yang, J. Park, A. Heinecke, E. Georganas, S. Srinivasan, A. Kundu, M. Smelyanskiy, B. Kaul, and P. Dubey, “A study of BFLOAT16 for deep learning training,” *arXiv preprint arXiv:1905.12322*, 2019.
- [10] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std. IEEE Std 754-2019, 2019.
- [11] J. R. Hauser, “Berkeley softfloat,” <https://www.jhauser.us/arithmetic/SoftFloat.html>, accessed: 2026-04-24.
- [12] S. Ul Haq, A. H. El-Maleh, and A. Alsuwaiyan, “Multiple-input floating-point adders: A comprehensive review,” *IEEE Access*, vol. 13, pp. 91 012–91 024, 2025.
- [13] Arm, “ULP Error and Accuracy,” <https://learn.arm.com/learning-paths/servers-and-cloud-computing/multi-accuracy-libamath/ulp-error/>, accessed: 2026-04-23.
- [14] Silicon Integration Initiative (Si2), “Nangate 45nm open cell library,” Available: <https://si2.org/open-cell-and-free-pdk-libraries/>, 2011.

A Teamwork

The project was initially split between an arithmetic team (Myles Querimit and Mixuan Pan) and an architecture team (Nikhil Vaidyanath, Emmi Muninnimit, Don Nguyen, and Seongjoong Yim). On the arithmetic side, Mixuan owned the BF16 multiplier and Myles owned the FP32 adder and BF16/FP32 accumulation logic, with both jointly verifying each unit using Berkeley Testfloat. The architecture, whose work is the primary focus of this report, team initially worked together on MEISSA; once the base version was functional, Emmi stayed on to design and integrate the additional adder-tree variations. Nikhil moved to the credit system shared across all three designs, and Don and Seongjoong shifted to the TPU-style design, with Seongjoong also leading the top-level test-benches to validate the 3 different architectures. Once the arithmetic units were complete, Myles joined the Standard architecture and Mixuan joined MEISSA to support integration and debugging. Synthesis was shared with Nikhil primarily taking ownership for the synthesis of the TPU and MEISSA architectures, and Myles and Mixuan for the Standard design and individual arithmetic units. The team coordinated through weekly meetings, a group chat for asynchronous discussion, and a shared Git integration branch where all work was merged. This work involved all team members individually writing many sections and reviewing the work of other team members.